Java Backend Mastery

The Definitive Interview Revision Guide

[Class, Objects, OOP, Exceptions]

1. CLASSES & OBJECTS

THE BLUEPRINT

A class is a template for creating objects. It defines the common properties and methods for a specific entity (e.g., Student).

```
class Student {
  String name;
  int marks;
}
```

THE REALIZATION

An object is an instance of a class. While the class defines structure, memory is only allocated when the object is instantiated.

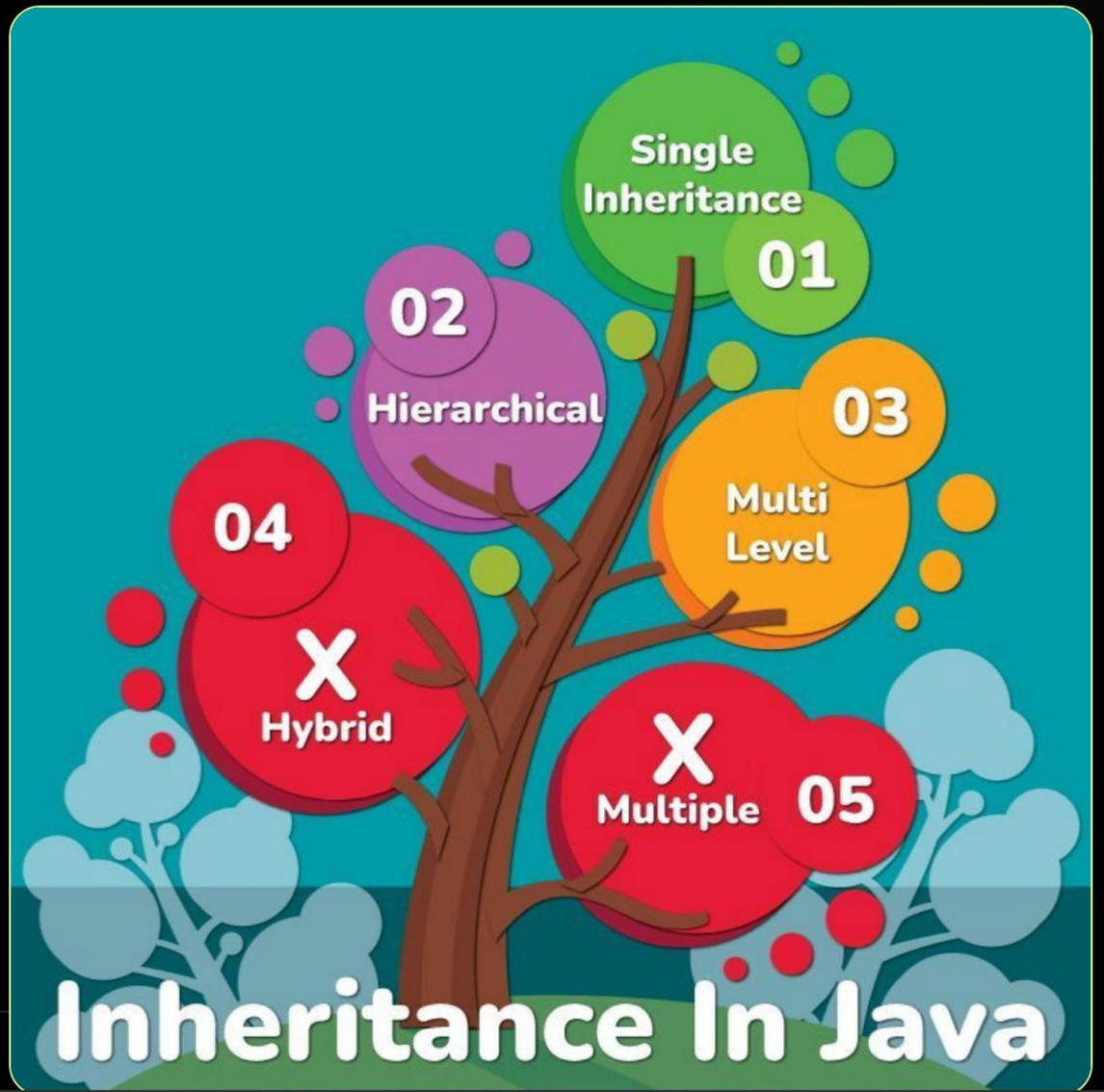
```
Student s1 = new Student();
s1.name = "Alice";
```

2. INHERITANCE: REUSABILITY

IS-A Relationship

- Allows a child class to acquire parent properties.
- Promotes code reuse and reduces redundancy.
- JVM links classes using the `extends` keyword.

```
class Animal {
  void eat() { System.out.println("Eating"); }
}
class Dog extends Animal { }
```



Polymorphism

"Many Forms" - Compile-Time vs Runtime

3. THE TWO FACES OF POLY

METHOD OVERLOADING

// Compile-Time Polymorphism

Multiple methods with same name but different parameters within the same class.

```
int add(int a, int b) { ... }
int add(int a, int b, int c) { ... }
```

METHOD OVERRIDING

// Runtime Polymorphism

Child class provides specific implementation of a method already in the parent class.

```
@Override
void pay() {
  System.out.println("Paid via UPI");
}
```

4. ENCAPSULATION: SECURE STATE

DATA HIDING

Variables are declared as `private` to prevent direct outside manipulation.

VALIDATION

Setters allow you to enforce rules (e.g., preventing a negative bank balance).

CONTROLLED ACCESS

Getters provide a safe, read-only window into the object's internal state.

```
private double balance;
public void deposit(double amount) { if(amount > 0) balance += amount; }
```

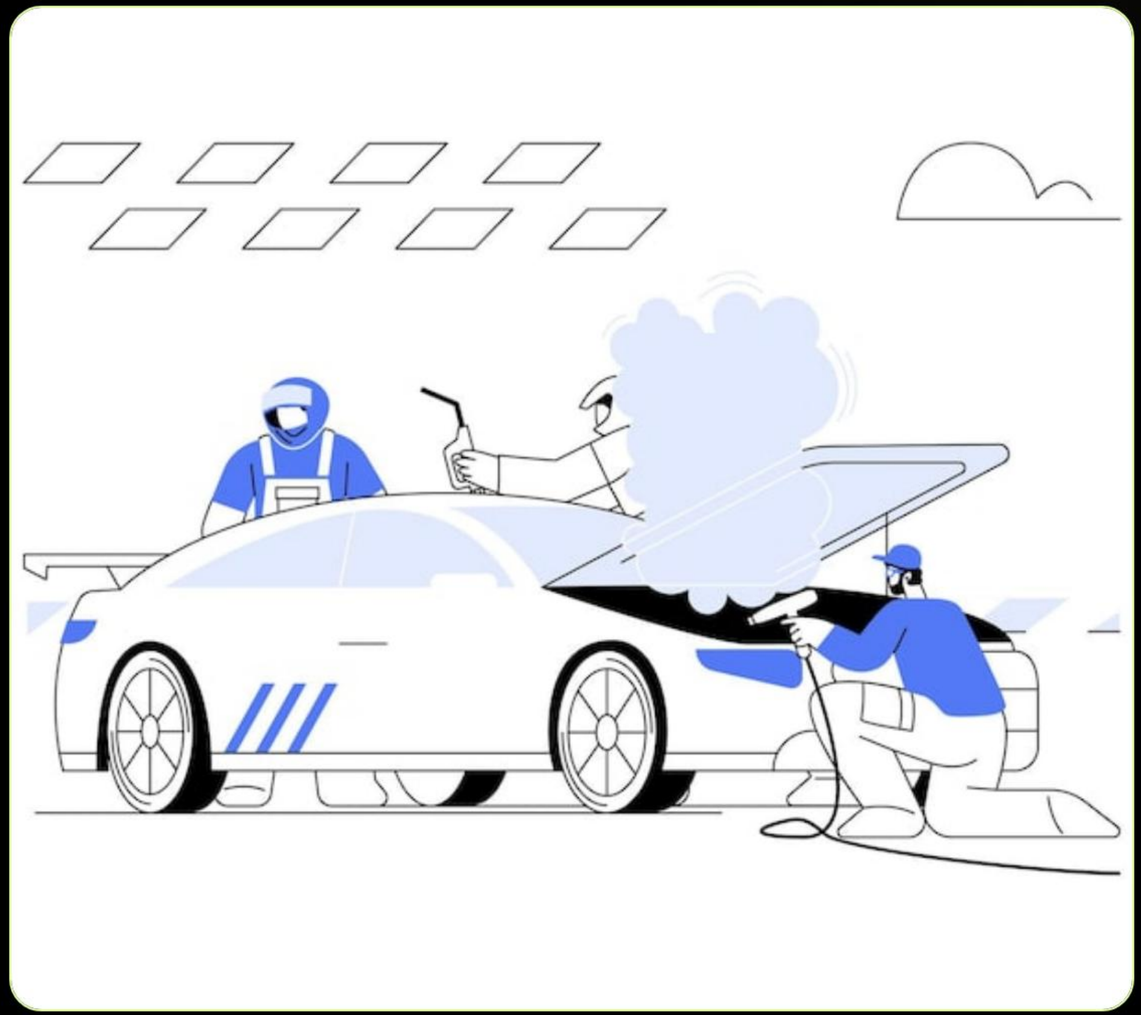
5. ABSTRACTION: FOCUS ON "WHAT"

Abstraction hides internal complexity and shows only essential features to the user.

Scenario: Car Acceleration

User calls `accelerate()`. They don't need to know if it's injecting petrol or spinning an electric motor.

```
abstract class Vehicle {
  abstract void accelerate();
}
```



6. CONSTRUCTORS & INITIALIZATION

JVM

Allocates Heap Space

- Special method to initialize objects.
- Same name as class, no return type.
- Called automatically on `new` keyword.
- Enforces required initial values.

```
class Student {
  String name;
  // Constructor
  Student(String name) {
    this.name = name;
  }
}
```

7. DESIGN: COUPLING VS COHESION

LOW COUPLING

Minimize dependencies between classes. Changes in Class A should not force changes in Class B.

HIGH COHESION

Every class should have a single, well-defined responsibility. Don't make "God Classes".

"Low coupling and high cohesion make a system flexible, scalable, and easy to maintain."

8. ABSTRACT CLASS VS INTERFACE

Feature	Abstract Class	Interface
Abstraction	Partial (Abstract + Concrete)	Full (100% Abstract*)
Methods	Any Access Modifier	Public by Default
Inheritance	Single (Extends)	Multiple (Implements)
Variables	Instance Variables allowed	Public Static Final only

*Java 8+ allows Default and Static methods in Interfaces.

9. JAVA POWER KEYWORDS

THIS

Refers to the current instance. Solves naming conflicts between parameters and instance variables.

SUPER

Refers to parent class. Used to call parent constructor or overridden methods.

FINAL

Constants. Prevents variables from changing, methods from overriding, or classes from extending.

Exception Handling

Maintaining the Normal Flow of Execution

10. STRICT VS SURPRISE

Checked (Strict)

Compiler forces you to handle these. Usually external issues like Disk or Network.

// IOException, SQLException

Unchecked (Runtime)

Flaws in logic. Compiler ignores these during build.

// NullPointerException, ArithmeticException

11. THROW VS THROWS

THROW (THE ACTION)

Used inside the method body to explicitly trigger an exception object.

```
throw new Exception("Error");
```

THROWS (THE WARNING)

Used in method signature to declare potential risks to the caller.

```
void read() throws IOException { ... }
```

12. EXCEPTION PROPAGATION

An unhandled exception travels **UP** the call stack until it finds a catch block.

```
main() → catches it ✅
  ↑
method3() → propagates
  ↑
method2() → propagates
  ↑
method1() → throws ❌
```

INTERVIEW TIP

Only **Unchecked** exceptions propagate by default. **Checked** exceptions must be declared via `throws` to propagate.

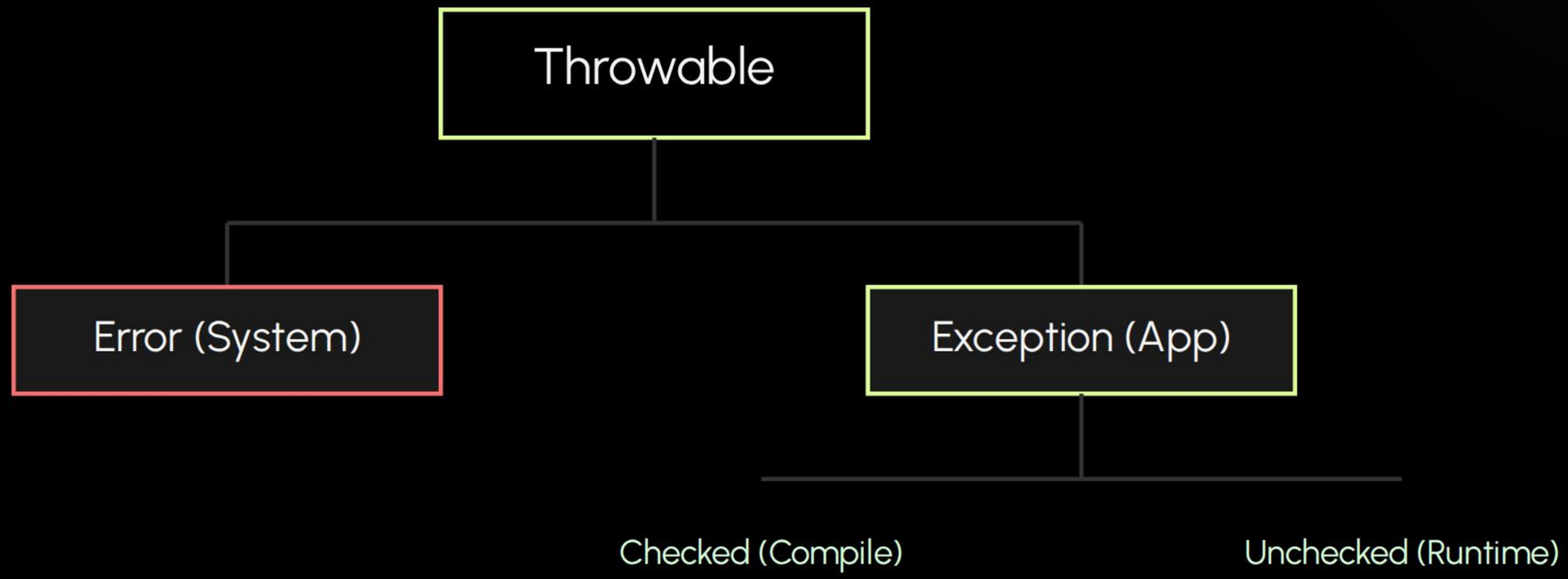
13. CUSTOM EXCEPTIONS

Why Subclass Exception?

- Specific error clarity for your app.
- Improves debugging and logging.
- Enhances readability of error messages.

```
class InvalidAge extends Exception {
  InvalidAge(String msg) {
    super(msg);
  }
}
```

14. THE THROWABLE HIERARCHY



Good Luck!

Master these concepts, and the offer is yours.

Java Core Revision | Interview Prep Series

IMAGE SOURCES

Thumbnail for dev.to

https://media2.dev.to/dynamic/image/width=1080,height=1080,fit=cover,gravity=auto,format=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2F7xq0zr7lto5exxz45b81.jpg

Source: dev.to

https://img.freepik.com/premium-vector/overheated-engine-abstract-concept-vector-illustration_107173-71044.jpg?semt=ais_wordcount_boost&w=740&q=80

Source: www.freepik.com

https://www.sentinelone.com/wp-content/uploads/2018/07/19175415/Java-Exceptions.png

Source: www.sentinelone.com
